# Group Project - Milestone 2

## Background

Congratulations! Thanks to your (tested) deliverables in **MS_1**, SpaceXXX managed to regain "manual control" over a very expensive exploration vehicle deployed in a ~~galax~~.. place, far, far away …

However, there are many issues, with using a human to :

- exert control "manually", issuing piecemeal instructions, which must then be …
- relayed via satellite, going thru multiple layers of communications equipment, to …
- finally reach the vehicle (while assuming "good" space/planetary weather conditions)

All of the above represent potential points of failure. (You only need 1 point to fail, and everything falls apart) And … perhaps most risky of all, SpaceXXX is also assuming the human in control, is "in-corruptible & un-bribable" … yes, using human could also be a point of failure!)

Your next challenge is to **evolve** your **MS_1** program, to include an "auto-pilot" function, which when activated, will enable the vehicle to make "intelligent decisions" regarding :

1) acquiring terrain info (e.g. which direction and how far to scan)
2) the most "energy cost" effective way to travel
3) which direction / how far to move
4) when and how much energy to allocate to shields

In other words, the auto-pilot function in your program should be able to determine **when** and **what sequence** of vehicle actions to perform, in the most *efficient* and/or *effective* way possible, so as to achieve different (pre-configured) mission objectives.

The next section describes the set of tasks / instructions for this MS_2 project.

## Task Instructions / Info

A) From the perspective of "front-end" terminal – keyboard user interaction, your team must include 2 more menu choices (in addition to those already implemented under MS_1) :

   (i) Configure Auto-Pilot display options (e.g. press 'o' key to configure …)

   (ii) Toggle start / stop Auto-Pilot function (e.g. press 'p' key to toggle on/off …)

B) Please refer to **Appendix A**, which provides further details with regards the kinds of options to be made available for user to configure (prior to activating the auto pilot function).

# APPENDIX A

( Configurable Auto-Pilot Options … )

As the Ubuntu OS terminal has a limited "vertical & horizontal" screen space, and considering how much "running output" could be generated when auto-pilot function is activated, it would be very difficult to keep track and verify what is happening if we simply let the huge amount of output overrun your entire display termimal!

Due to above display-related issues, all teams are to provide the following options, that allows the user to choose the kinds of output to focus on, and/or how long to pause (after different types of info display), so that a user like Agent 'T' can track what is happening with the vehicle, and what kind of vehicular "auto-pilot" actions it is currently taking …

These options are :

1. After every action, show vehicle status / energy stats (yes/no?)
2. After every scan action, show vehicle status / energy stats (yes/no?)
3. After every move action, show vehicle status / energy stats (yes/no?)
4. After every shield allocation, show vehicle status / energy stats (yes/no?)
5. After every scan action, display 'current map' of terrain scanned by vehicle so far (yes/no?)
6. After every move action, display 'current map' of terrain travelled by vehicle so far (yes/no?)
7. Enter '`t1`' secs delay, after displaying vehicle status / energy stats : _____
8. Enter '`t2`' secs delay, after displaying 'current map' of terrain scanned : _____
9. Enter '`t3`' secs delay, after displaying 'current map' of terrain travelled : _____
10. Enter '`t4`' secs delay, after any action taken by auto-pilot function : _____
11. Enter '`t5`' secs delay, after each scan action taken by auto-pilot function : ___
12. Enter '`t6`' secs delay, after each move taken by auto-pilot function : _____
13. Enter '`t7`' secs delay, after each shield allocation action taken by auto-pilot function : _____

**Note :**

Option 1 :
if yes, will override Options 2 – 4 (i.e. if yes, then no need to display/prompt with options 2 – 4)
if no, then prompt with further options 2 – 4

Option 10 :
if ('t4' > 0), will override Options 11 – 13 (i.e. no need display/prompt with options 11 – 13)
if ('t4' == 0) then prompt with further options 11 – 13

Finally, when auto-pilot function is activated, it should (ideally) keep running without further user intervention, unless the conditions to pause / stop the auto-pilot function is met.

In order to configure such stopping / pausing criteria, all teams are to provide the following related options as well :

a) Pause/stop auto-pilot when current energy available falls below : __ units

b) Pause/stop auto-pilot when total no. of move actions exceed : ____ steps

c) Pause/stop auto-pilot when total no. of scan actions exceed : _____ scans

d) Pause/stop auto-pilot when mission objective is achieved (i.e. if mission is :

- "casual explore" – stop when any of the earlier conditions a) – c) are met
- "find 1 end point" – stop the moment your scans reveal 1 end point location (e.g. 'E' symbol)
- "reach 1 end point" – stop the moment your vehicle move to, and occupy 1 end point location
- "find all end points" – stop the moment your auto-pilot function "believes" it has scanned and located all the end point locations!
- "reach all end points" – stop the moment your auto-pilot function "believes" it has travelled to and occupied all the end points locations (in its history of travels) !
- "find all danger points" – stop the moment your auto-pilot function "believes" it has scanned and located all the danger point locations (e.g. 'X' symbol)
- "find map boundaries" – stop the moment your auto-pilot function "believes" it has scanned and located all locations out-lining the perimeter of the map (e.g. '#' symbol)
- "map everything" – stop the moment your auto-pilot function "believes" it has scanned and located the ENTIRE MAP of terrains